



Robótica:
Relatório de implementação do Método da Janela Dinâmica

Discente: Adhemar Luiz Cavalcanti Silva Rocha Neto
Docente: Juracy Emanuel Magalhães

1. Introdução

Este relatório documenta a implementação de um sistema de navegação autônoma para um robô diferencial em ambiente simulado. A solução utiliza o algoritmo **Dynamic Window Approach (DWA)** para realizar o planejamento de trajetória local, permitindo que o robô se desloque até um ponto de destino enquanto desvia dinamicamente de obstáculos detectados em tempo real por sensores ultrassônicos.

2. Arquitetura de Software

O projeto foi desenvolvido seguindo uma estrutura modular para garantir a separação entre a inteligência algorítmica e a interface de hardware (simulador):

- **config_dwa.py**: Contém as restrições físicas do robô (velocidades e acelerações) e os pesos da função de custo (prioridade entre velocidade, alvo e desvio).
- **dwa_logic.py**: Implementa o motor de cálculo do DWA, que simula trajetórias futuras e seleciona a de menor custo.
- **main_robo.py**: Script principal que gerencia o ciclo de controle, conectando-se ao Coppeliasim para leitura de dados e atuação nos motores.

3. Implementação da Percepção

O ponto central do sucesso deste projeto reside na camada de percepção. O robô não possui um mapa prévio do ambiente; ele precisa "construir" uma consciência espacial a cada centésimo de segundo.

3.1. Transformação de Referenciais (Local para Global)

O robô está equipado com **5 sensores de proximidade**. O desafio técnico superado foi que cada sensor informa a distância de um obstáculo em relação a si mesmo (Referencial Local). Para que o robô tome decisões de desvio no mapa da sala, esses dados brutos precisam ser convertidos para o Referencial Global.

O processo funciona da seguinte forma:

1. **Leitura do Sensor**: O sensor detecta um ponto de impacto P_{local} a uma distância d .
2. **Matriz de Transformação**: O sistema extrai a Matriz de Transformação Homogênea M do sensor. Esta matriz contém a posição $[x, y, z]$ e a orientação α, β, γ exata do sensor no mundo.
3. **Cálculo Vetorial**: Aplicando a operação:
$$P_{global} = M\{sensor\} \times P_{local}$$

Isso converte o ponto relativo "na frente do sensor" em uma coordenada fixa X, Y no mapa do ambiente de simulação.

3.2. Mapeamento de Obstáculos em Tempo Real

Ao final de cada varredura dos 5 sensores, o script gera uma lista de coordenadas de obstáculos. Esta lista é enviada para a lógica do DWA, que passa a enxergar esses pontos

como "zonas de colisão", forçando o robô a buscar velocidades que geram trajetórias que contornam tais coordenadas.

4. Lógica de Decisão e Navegação

A lógica do robô segue um ciclo contínuo de **Percepção-Decisão-Ação**:

1. **Geração de Trajetórias**: O software simula dezenas de curvas possíveis que o robô pode realizar fisicamente.
2. **Avaliação de Segurança**: O sistema cruza os dados da Percepção (os pontos dos obstáculos) com as trajetórias simuladas. Qualquer caminho que colida com um obstáculo dentro do `robot_radius` (raio de segurança) é descartado.
3. **Seleção Ótima**: Entre os caminhos seguros, o software seleciona aquele que:
 - Mais se aproxima da direção do alvo (**Custo de Alvo**).
 - Mantém a maior velocidade possível (**Custo de Velocidade**).

5. Resultados Experimentais

Durante os testes no CoppeliaSim, os seguintes marcos de sucesso foram atingidos:

- **Conexão de Baixa Latência**: A integração via ZMQ permitiu um ciclo de controle estável de 10Hz (0.1s), ideal para robótica reativa.
- **Desvio Dinâmico**: Ao inserir obstáculos primitivos (caixas e paredes) no ambiente, o robô demonstrou capacidade de detecção imediata, alterando sua velocidade angular omega para contornar o perigo.
- **Retorno à Rota**: Uma vez superado o obstáculo, a lógica de custo de alvo reconduziu o robô automaticamente para as coordenadas de destino final (5, 5).
- **Cercamento de Segurança**: A implementação de paredes perimetrais configuradas como `Detectable` e `Collidable` garantiu que o robô permanecesse dentro dos limites do cenário.

6. Conclusão

A implementação provou ser robusta para a tarefa de navegação autônoma local. O diferencial deste trabalho foi a correta tradução geométrica dos sensores, permitindo que uma lógica abstrata (DWA) operasse sobre um hardware simulado com precisão. O sistema resultante é capaz de navegar em ambientes desconhecidos de forma totalmente autônoma e reativa.

Link para acesso ao github: https://github.com/AdhemarCavalcanti/Rob-tica_2026.1